Intro. To Formal Verification

Presentation: DPLL Algorithms & z3 Solvers

Team : Hitanshu & Hemang

Flow Of Presentation

01 Problems With Conventional Algo's

Issues with basic recursive logics and their Exponential Time

02 DPLL Algorithm explanation

Going through the DPLL algorithm and its explanation

03 Example and different use cases

Showing the usage of DPLL across different examples

04 z3-Solver Assignment (Application of DPLL)

Walk through the z3-Solver Assignment Exercise 2

Problem With Conventional Algorithms

- **Every propositional formula with n variables has exactly $O(2^n)$possible truth assignments. A naive solver must potentially check all of them .**
- **Brute force is not just slow — it is physically impossible.**

Three Points Of Failures In Brute Force Algorithm →

1. **Failure 1 No Early Termination**
   Picks any variable even if any individual clause is causing it to explore 2^{n-1} branches
2. **Failure 2 Blind Variable Selection**
   Wrong variable selected means a lot of calculations
3. **Failure 3 Redundant Conflict Re-discovery**
   No memory, it keeps learning the same pattern without stopping on backtracking

BRUTE FORCE ALGORITHM

```
p = ChooseVariable(F) // picks any variable,
no strategy try p = True:

for every remaining variable q: try q = True: for every remaining variable r: ... // full binary tree, no shortcuts
```

What is DPLL, Conversion to CNF

- **DPLL** is a **complete algorithm** - it always gives the right answer
- **It is sound** - if it says SAT, the model it returns actually satisfies F
- It decides SAT for any CNF formula in **finite time**
- It is the **backbone** of every modern solver (Z3, MiniSAT, CryptoMiniSat)
- The name: **Davis, Putnam, Logemann, Loveland** - published 1962

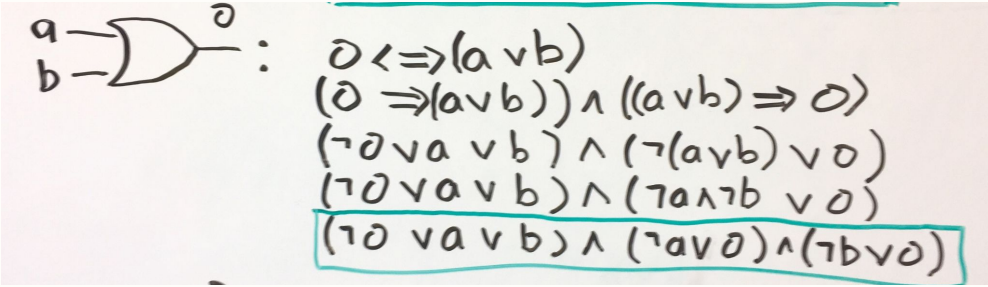
F = {C1, C2, C3, ...} ← a SET of clauses (all must be satisfied)

Each clause Ci = {l1, l2, ...} ← a SET of literals (at least one must be true)

A literal is either:

x (x must be True)

¬x (x must be False)



Step 1: Unit Propagation

The Core Idea →

- A unit clause has exactly one literal.
- That literal has **NO CHOICE** it must be True.
- No branching. No guessing. Pure logical force.

PseudoCode Of the Step-1

```
function simplify(F, literal l):
   new_F = {}
   for each clause C in F:
      if l ∈ C:
         skip C          // clause is satisfied, remove it
      else:
         new_F.add(C \ {¬l})   // remove the negation
of l from clause
   return new_F
```

Unit propagation runs in **linear time O(|F|) per pass**

One forced assignment often triggers a chain of forced assignments called **Ripple Effect**

If simplification produces an empty clause {} → contradiction → UNSAT on this branch

In practice, **unit propagation alone solves a large fraction of industrial SAT instances**

Step 2: Pure Literal Elimination

- A literal l is PURE if its negation ¬l never appears anywhere in F.
- Assigning l = True satisfies every clause containing l.
- Since ¬l doesn't exist, this assignment breaks nothing elsewhere. It never causes a contradiction.

Proof Of Correctness

If l is pure and we set l = True:

Every clause containing l is satisfied (good)

No clause contains ¬l so nothing is broken (safe)

A solution WITH l=True exists iff a solution exists at all

Pure literal elimination **runs in O(|F|)**

Together, UP + PLE form the **inference engine of DPLL**

UP + PLE together empty the formula → SAT without ever branching

Real formulas from **hardware verification** **are often solved entirely by these chains**

Step 3: Branching & MOM's Heuristic Part-1

After running UP and PLE, if F still has clauses remaining:

→ No unit clauses exist
→ No pure literals exist

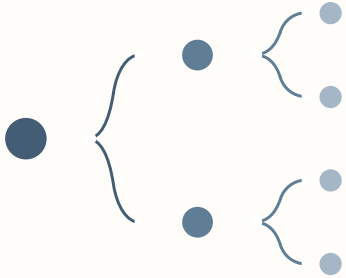
This is the branching step → We pick one variable, try True, try False, recurse into both.

IMPORTANCE OF CHOOSING CORRECT

Choose those that appears in many SHORT clauses
→ resolves **multiple clauses** at once
→ triggers unit propagation chain immediately
→ tree stays **shallow**

```
function(F):
    k = minimum clause length in F // find the tightest constraints
    S = all clauses in F of length k // these are closest to becoming units

    for each variable x:
        score(x) = (cnt of x in S) + (cnt of ¬x in S)
    return variable with highest score
```



Step 3: Branching & MOM's Heuristic Part-2

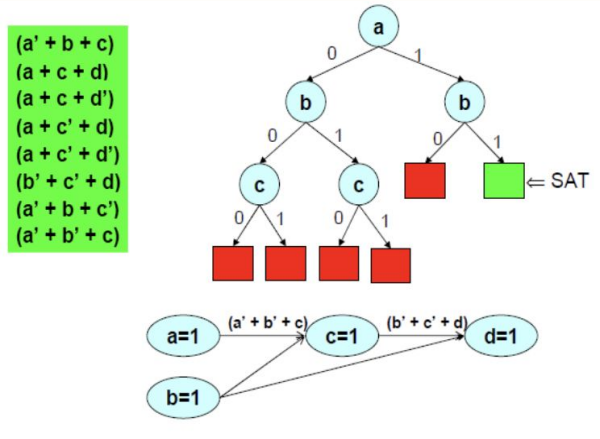
Ripple Effect →

- A single well-chosen branch on a can trigger a chain of forced **Unit Propagations** one guess collapses multiple clauses simultaneously, turning a complex formula into a trivial one in just a few steps.
- If one branch hits a contradiction, the other branch inherits a smaller, simplified formula the work done during propagation is **never wasted,** it always narrows down the remaining search space regardless of which side succeeds.

- **Branching** is the only **non-deterministic** step in DPLL
- A good heuristic can reduce the tree from 2^n to $2^{n/10}$ in practice
- MOM's is simple but effective modern solvers use **VSIDS** (Variable State Independent Decaying Sum) which learns from conflict history
- Every branch is an **independent subproblem** if one side returns SAT we are done immediately

Complete DLL Algorithm

```
function DPLL(F, α):
  if F = ∅        → return SAT, α
  if {} ∈ F       → return UNSAT          // Phase 1: Base Cases
  while ∃ unit (l) in F:
    α = α ∪ {var(l) → pol(l)}
    F = simplify(F, l)                    // Phase 2: Unit Propagation
    if {} ∈ F     → return UNSAT
  for each pure l in F:
    α = α ∪ {var(l) → pol(l)}             // Phase 3: Pure Literal Elimination
    F = simplify(F, l)
  if F = ∅        → return SAT, α
  if {} ∈ F       → return UNSAT          // Phase 4: Re-check Base Cases
  x = ChooseVar(F)
  if DPLL(simplify(F, x),  α∪{x→T})=SAT
        → return SAT                      // Phase 5: Heuristic Branching
  if DPLL(simplify(F, ¬x), α∪{x→F})=SAT
        → return SAT
  return UNSAT
```



```
function DPLL(F, α):
```

PHASE 1 — BASE CASES

```
if F = ∅          → return SAT, α        SAT
if {} ∈ F         → return UNSAT         UNSAT
```

PHASE 2 — UNIT PROPAGATION

```
while ∃ unit (l) in F:                   FORCED
  α = α ∪ {var(l) → pol(l)}
  F = simplify(F, l)
  if {} ∈ F     → return UNSAT           UNSAT
```

PHASE 3 — PURE LITERAL ELIMINATION

```
for each pure l in F:                    FREE
  α = α ∪ {var(l) → pol(l)}
  F = simplify(F, l)
```

PHASE 4 — RE-CHECK BASE CASES

```
if F = ∅          → return SAT, α        SAT
if {} ∈ F         → return UNSAT         UNSAT
```

PHASE 5 — HEURISTIC BRANCHING

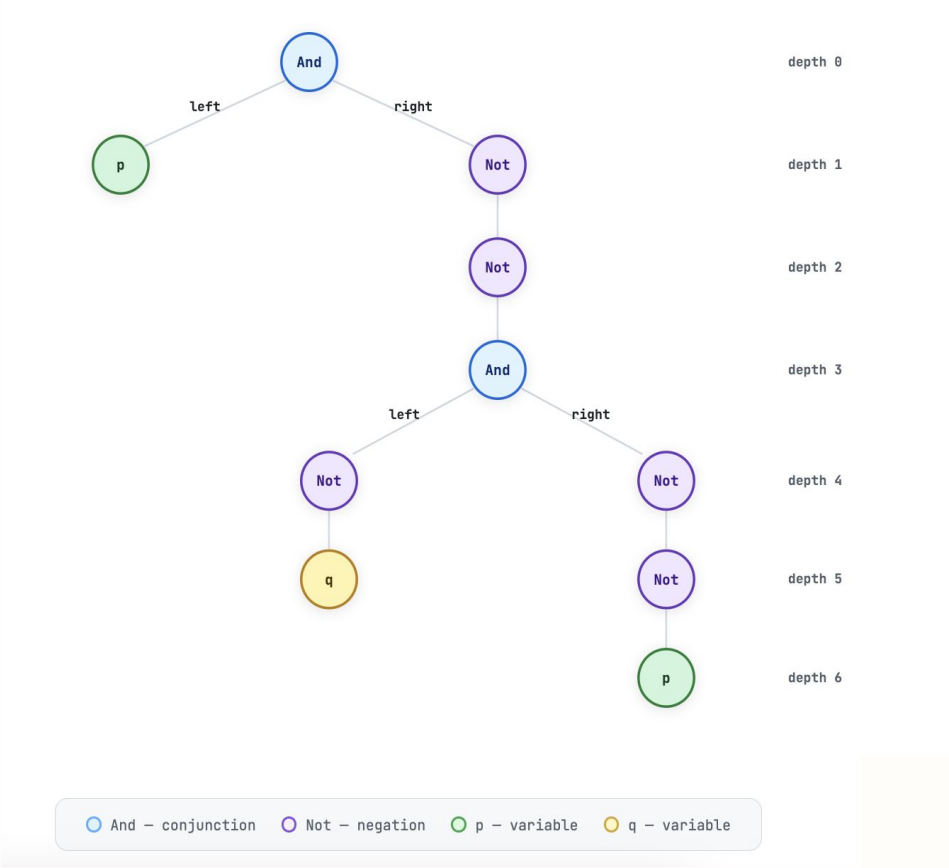
```
x = ChooseVar(F)       // MOM's heuristic                    PICK
if DPLL(simplify(F, x), α∪{x→T}) = SAT  → return SAT          x = T
if DPLL(simplify(F, ¬x), α∪{x→F}) = SAT  → return SAT         x = F
return UNSAT             // both branches failed              UNSAT
```

Base Cases · Unit Propagation · Pure Literal Elim. · Re-check · Heuristic Branch · SAT terminal · UNSAT terminal

What is z3 Solver ?

- Z3 is an SMT solver (Satisfiability Modulo Theories) built by Microsoft Research
- SMT extends SAT — it can reason about not just Boolean logic but also integers, arrays, strings, and real arithmetic
- For pure propositional logic like ours, Z3 uses its CDCL engine internally — the same DPLL-based algorithm, but with clause learning
- Used in industry: Azure cloud verification, Windows kernel checking, compiler analysis
- Python API via from z3 import * — one import gives access to the full solver

Our Assignment & Work



- Parses any formula you type parse_formula() recursively splits on ¬, ∧, ∨ in correct precedence order and builds a Z3 expression tree on the fly.
- Auto-discovers variables dynamically no hardcoded Bool('p') anywhere; every new variable name found gets created automatically, so it works for any formula.
- Calls Z3 and returns the result — s.check() runs the CDCL engine and either returns SAT with a model like [p = True, q = False] or proves the UNSAT.

```
    # Handle ¬ (negation) at the start
    if expr.startswith('¬') or expr.startswith('~'):
        return Not(parse_formula(expr[1:].strip(), variables))

    for op_sym, op_fn in [('∨', Or), ('∧', And)]:
        depth = 0
        for i in range(len(expr) - 1, -1, -1):
            if expr[i] == ')': depth += 1
            elif expr[i] == '(': depth -= 1
            elif depth == 0 and expr[i] == op_sym:
                left  = expr[:i].strip()
                right = expr[i+1:].strip()
                return op_fn(parse_formula(left, variables),
                             parse_formula(right, variables))

    # It's a variable
    name = expr.strip()
    if name not in variables:
        variables[name] = Bool(name)
    return variables[name]


def extract_variables(expr):
    return sorted(set(re.findall(r'\b[a-zA-Z_]\w*\b', expr)))
```

Thank You